Documento de Arquitectura — Sistema SmartPlantBase (Fase 2)

Proyecto Matera Inteligente · Arquitectura Hexagonal (Puertos y Adaptadores) · **Estudiante:** Esteban Castro · **Semestre:** 2026-3 · **Tecnologías:** Python 3.14, Flask, SQLite, PyJWT, pytest

1. Diagrama Hexagonal del Sistema

Para la segunda fase de SmartPlantBase estructuramos la aplicación mediante el patrón de **Puertos y Adaptadores (Alistair Cockburn, 2005)**. El objetivo principal es aislar la lógica botánica y las reglas de negocio de cualquier detalle tecnológico: protocolos de transporte, frameworks web, controladores de bases de datos o mecanismos de serialización.

La Figura 1 ilustra la vista de referencia de nuestra arquitectura, reflejando con exactitud los paquetes, clases, puertos y adaptadores implementados en el código:

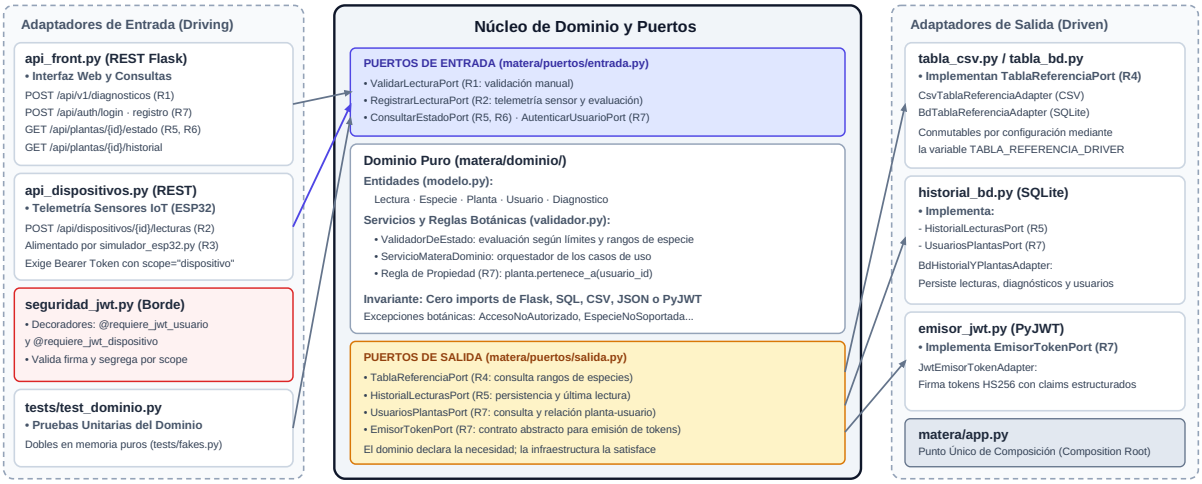


Figura 1. Vista hexagonal del sistema con los adaptadores, puertos y el núcleo de dominio implementados en el proyecto.

El flujo de dependencias apunta siempre hacia adentro:

- **Adaptadores de entrada (primarios):** Traducen estímulos del exterior a invocaciones de los puertos de entrada. El módulo `api_front.py` maneja las solicitudes HTTP web del usuario y `api_dispositivos.py` recibe la telemetría periódica del ESP32.
- **Núcleo de dominio:** Agrupa las entidades botánicas (`Lectura` , `Especie` , `Planta` , `Usuario` , `Diagnostico`) y los servicios de evaluación (`ValidadorDeEstado`). No contiene librerías externas ni referencias a HTTP o SQL.
- **Adaptadores de salida (secundarios):** Implementan los contratos definidos por el dominio para interactuar con la infraestructura: almacenamiento relacional en SQLite (`historial_bd.py`), catálogo botánico (`tabla_csv.py` y `tabla_bd.py`) y firma de credenciales (`emisor_jwt.py`).

2. Tabla de Puertos y Adaptadores

Los puertos desacoplan el núcleo de las tecnologías externas mediante contratos explícitos definidos en Python con clases base abstractas (`abc.ABC`):

Puerto	Dirección	Archivo de Interfaz	Adaptador Concreto	Responsabilidad en el Negocio
ValidarLecturaPort	Entrada	`matera/puertos/entrada.py`	ServicioMateraDominio *(consumido por `api_front.py`)*	Evalúa lecturas enviadas de forma manual desde la interfaz web contra los rangos de la especie (R1).
RegistrarLecturaPort	Entrada	`matera/puertos/entrada.py`	ServicioMateraDominio *(consumido por `api_dispositivos.py`)*	Procesa la telemetría enviada por el ESP32 (R2), calcula el diagnóstico botánico y solicita la persistencia histórica (R5).
ConsultarEstadoPort	Entrada	`matera/puertos/entrada.py`	ServicioMateraDominio *(consumido por `api_front.py`)*	Recupera el estado actual y el historial de una planta, aplicando la validación de propiedad de usuario (R5, R6, R7).
AutenticarUsuarioPort	Entrada	`matera/puertos/entrada.py`	ServicioMateraDominio *(consumido por `api_front.py`)*	Autentica credenciales de usuarios, registra cuentas nuevas y coordina la generación de tokens de sesión (R7).
TablaReferenciaPort	Salida	`matera/puertos/salida.py`	CsvTablaReferenciaAdapter BdTablaReferenciaAdapter	Proporciona las especies botánicas y sus rangos óptimos. Es intercambiable por configuración (CSV o SQLite) (R4).
HistorialLecturasPort	Salida	`matera/puertos/salida.py`	BdHistorialYPlantasAdapter	Persiste cada lectura recibida junto con su diagnóstico y consulta el registro más reciente por planta (R5).
UsuariosPlantasPort	Salida	`matera/puertos/salida.py`	BdHistorialYPlantasAdapter	Almacena y consulta la información de usuarios, plantas registradas y la relación entre sensor y planta (R7).
EmisorTokenPort	Salida	`matera/puertos/salida.py`	JwtEmisorTokenAdapter	Firma tokens criptográficos JWT segregados para usuarios y dispositivos mediante el algoritmo HS256 (R7).

3. Manejo de Seguridad, Alcances de JWT y Regla de Propiedad

La autenticación y autorización protegen el sistema en el perímetro sin invadir el núcleo del dominio:

- **Segregación de Alcances (User vs Device):** El sistema emite dos tipos de tokens disyuntos mediante el claim `scope` . El **token de usuario** (`scope="usuario"`), emitido en `/api/auth/login` , autoriza la consulta y gestión de plantas propias. El **token de dispositivo** (`scope="dispositivo"`), asignado al sensor ESP32, autoriza únicamente el reporte de telemetría en `POST /api/dispositivos/{id}/lecturas` .
- **Puntos de Emisión y Verificación:** La emisión se delega al puerto de salida `EmisorTokenPort` , implementado por `JwtEmisorTokenAdapter` en `matera/adaptadores/salida/emisor_jwt.py` (único punto que utiliza `PyJWT` con algoritmo HS256). La verificación ocurre en los adaptadores de entrada mediante los decoradores `@requiere_jwt_usuario` y `@requiere_jwt_dispositivo` (`seguridad_jwt.py`), rechazando con HTTP 403 Forbidden cualquier intento de cruzar alcances antes de invocar al dominio.
- **Aislamiento del Dominio:** El dominio desconoce el estándar RFC 7519 y las cabeceras HTTP `Authorization` . El decorador extrae la identidad limpia (`usuario_id` o `dispositivo_id`) y la entrega como parámetro al caso de uso, permitiendo probar el dominio de forma aislada sin generar tokens ni depender de claves criptográficas.
- **Regla de Propiedad en el Dominio:** La verificación de pertenencia es una regla botánica y reside en el dominio:

```
# matera/dominio/validador.py (ConsultarEstadoPort)
planta = self._usuarios_plantas.obtener_planta_por_id(planta_id)
if not planta.pertenece_a(usuario_id):
    raise AccesoNoAutorizadoPlantaError(usuario_id=usuario_id, planta_id=planta_id)
```

Si un usuario autenticado intenta consultar una planta que no le pertenece, el dominio lanza `AccesoNoAutorizadoPlantaError` , y la capa HTTP responde con HTTP 403 Forbidden (`ACCESO_DENEGADO`).

4. Justificación de Principios SOLID en el Código

La estructura de paquetes y clases del proyecto aplica los principios de diseño orientado a objetos:

- **Inversión de Dependencias (DIP):** El núcleo de dominio no depende de bibliotecas externas ni de adaptadores concretos; depende de abstracciones. `ServicioMateraDominio` recibe la interfaz `TablaReferenciaPort`, por lo que desconoce si las especies provienen de un archivo de texto o de SQLite:

```
# matera/puertos/salida.py
class TablaReferenciaPort(ABC):
    @abstractmethod
    def obtener_por_nombre(self, nombre: str) -> Optional[Especie]:
        """Contrato para consultar especie sin acoplarse al medio de persistencia."""
```

Tanto `CsvTablaReferenciaAdapter` como `BdTablaReferenciaAdapter` implementan esta abstracción. El módulo `validador.py` no importa `sqlite3` ni `csv`.

- **Abierto/Cerrado (OCP):** El sistema permite incorporar nuevas funcionalidades mediante la adición de adaptadores, manteniendo el dominio cerrado a modificaciones. Al añadir la telemetría del ESP32 (R2) y el simulador (R3), únicamente construimos `api_dispositivos.py` y `simulador_esp32.py`; el evaluador botánico no requirió ajustes. Igualmente, alternar entre CSV y base de datos (R4) se resuelve en el Composition Root (`matera/app.py`) sin tocar el dominio.
- **Segregación de Interfaces (ISP):** Definimos puertos específicos en lugar de interfaces genéricas sobrecargadas. `TablaReferenciaPort` contiene únicamente lectura de especies botánicas; `HistorialLecturasPort` expone persistencia de telemetría; `EmisorTokenPort` define la generación de tokens. Los adaptadores de entrada solo conocen el puerto que requieren para su caso de uso (por ejemplo, `api_dispositivos.py` solo recibe `RegistrarLecturaPort`).
- **Responsabilidad Única (SRP):** Cada componente tiene una única razón de cambio. `ValidadorDeEstado` se modifica únicamente si cambian las reglas botánicas de evaluación; `seguridad_jwt.py` cambia si varía la política de autenticación HTTP; y `historial_bd.py` cambia si se altera el esquema relacional de SQLite.

---

5. Verificación de la Prueba de Cambio: Sensor ESP32

Para verificar que la arquitectura permite integrar nuevos canales sin perturbar las reglas de negocio, la incorporación del sensor ESP32 y su simulador se realizó en un commit dedicado:

- **Identificador de Commit:** `8c4b6d8685aeb2bf2784258d349550a6cbf25812`
- **Mensaje del Commit:** `feat(dispositivos): adaptador de dispositivos y simulador ESP32 (Prueba de Cambio)`
- **Archivos creados y modificados:**
  - `matera/adaptadores/entrada/api_dispositivos.py` (nuevo adaptador REST para el sensor)
  - `simulador_esp32.py` (script autónomo de telemetría y pruebas de robustez)
  - `tests/test_api_dispositivos.py` (suite de integración para el nuevo endpoint)
  - `matera/app.py` (registro del blueprint en el punto de composición)

Comprobación de Invarianza del Dominio

Al auditar los cambios introducidos entre dicho commit y su predecesor sobre las carpetas de negocio:

```
git diff 8c4b6d8~1 8c4b6d8 -- matera/dominio/ matera/puertos/
```

El resultado no reporta ninguna diferencia. Los archivos `modelo.py`, `validador.py`, `excepciones.py`, `entrada.py` y `salida.py` no sufrieron alteraciones.

Esto confirma que el requerimiento R2 se conectó utilizando el puerto preexistente `RegistrarLecturaPort`, demostrando que la separación entre adaptadores y dominio preserva la estabilidad del núcleo cuando se añaden nuevas fuentes de datos.

6. Análisis de Pregunta de Cambio: Migración de Transporte a MQTT

Durante la sustentación se evalúa la capacidad de evolución del sistema ante un cambio tecnológico imprevisto:

> *"Si mañana las lecturas ambientales de la matera ya no llegaran por solicitudes HTTP REST, sino mediante mensajes asíncronos publicados en un broker MQTT (por ejemplo, en el tópico `matera/dispositivos/{id}/telemetria`), ¿qué componentes del repositorio cambiarían y cuáles permanecerían intactos?"*

Componentes Nuevos a Implementar (Adaptador de Entrada)

1. **Nuevo adaptador de suscripción MQTT (`matera/adaptadores/entrada/suscriptor_mqtt.py`):**
   - Utiliza una librería de cliente MQTT como `paho-mqtt`.
   - Se suscribe al patrón de tópicos `matera/dispositivos/+/telemetria`.
   - Al recibir un mensaje en su callback `on_message`, extrae el identificador del dispositivo del tópico, deserializa el payload JSON y valida la integridad de los campos (`humedad`, `luz`, `temperatura`, `marca_tiempo`).
   - Invoca directamente el puerto de entrada:

```
diagnostico = self._puerto_registro.registrar(
    dispositivo_id=dispositivo_id,
    valores=valores_medidos,
    marca_tiempo=marca_tiempo,
)
```

1. **Suite de pruebas del adaptador (`tests/test_suscriptor_mqtt.py`):**
   - Verifica la recepción de mensajes, el manejo de payloads corruptos y la correcta llamada a `RegistrarLecturaPort` mediante dobles de prueba.

Componentes Existentes a Modificar (Únicamente Composición y Entorno)

1. **Punto único de composición (`matera/app.py`):**
   - Se instancia el nuevo adaptador `suscriptor_mqtt` pasándole la misma instancia existente de `servicio_dominio` (que implementa `RegistrarLecturaPort`).
   - Se inicia el bucle de red del cliente MQTT en segundo plano (`client.loop_start()`).
2. **Dependencias del proyecto (`requirements.txt`):**
   - Se añade la dependencia del cliente MQTT (`paho-mqtt`).

Componentes que Permanecen Intactos

- **Núcleo de dominio (`matera/dominio/*`):** `modelo.py`, `validador.py` y `excepciones.py` no requieren ningún cambio. El evaluador botánico desconoce por completo si las lecturas provinieron de un paquete TCP, un socket MQTT o una petición HTTP.
- **Puertos de entrada y salida (`matera/puertos/*`):** `RegistrarLecturaPort` conserva su firma agnóstica (`dispositivo_id: str`, `valores: Mapping`, `marca_tiempo: str`), ya que fue diseñado pensando en el caso de uso y no en el protocolo.
- **Adaptadores de salida (`matera/adaptadores/salida/*`):** La persistencia del historial en SQLite (`historial_bd.py`) y la consulta de rangos de especies (`tabla_csv.py` / `tabla_bd.py`) se ejecutan exactamente igual.

---

7. Conclusión y Síntesis de la Arquitectura

La adopción de arquitectura hexagonal en esta segunda fase de SmartPlantBase resuelve dos problemas fundamentales de la ingeniería de software: el desacoplamiento de la tecnología frente a las reglas botánicas y la segregación limpia de responsabilidades de seguridad. Al ubicar la verificación de tokens JWT en los adaptadores primarios y delegar la regla de propiedad al dominio, logramos un sistema robusto, fácil de someter a pruebas unitarias en memoria y preparado para incorporar futuros clientes móviles o sensores industriales sin deuda técnica.